

CSE 144 Final Project Report

Transfer Learning Challenge Spring 2026

Elina Wilson (CruzID: 2043913)
Piet Rottinghuis (CruzID: 2022928)

1 Introduction:

The purpose of this project is to design a transfer learning approach for an image classification machine learning model. Numerous models have been developed and trained for image classification tasks. The weights learned by these models can be reused for similar image classification tasks, as the deeper layers of these models tend to learn generic image features and patterns. These features and patterns can be utilized across datasets. We can leverage these pretrained weights to reduce computational training cost and focus computational resources on optimizing performance for our specific dataset.

We utilized the EfficientNetV2-S, which was designed for image classification tasks [1]. Using the default pretrained weights distributed by torchvision, we initialized our model. During training, all layers of the model were fine-tuned using low learning rate and weight decay. We also increased the effective size and diversity of the training data through data augmentation.

2 Dataset

The dataset used for this project contains 2,115 images of cars, planes, flowers, and food distributed across 100 different categories. Training images number 1,079, with approximately 10 images in each category. The images vary in size and are all in color. Of the training images, 20% (approximately two) images in each category are reserved for validation.

When downloaded, the dataset is organized into two directories: “train” and “test”. The “test” directory contains numbered images whose classification labels must be predicted by the model. The “train” directory contains numbered subdirectories (1-100), where each subdirectory corresponds to a class label. As mentioned above, the “train/1-100” directories contain about 10 images each that are used for model training.

In addition to the base labeled image dataset, our approach incorporates several data augmentation techniques. These include image flipping, rotation, cropping, color jittering, greyscale conversion, random erasing, CutMix, and MixUp. These augmentations increase diversity of training data, which helps reduce overfitting and improve the models ability to generalize to unseen data.

3 Implementation

3.1 Model

The strongest model evaluated in this project is EfficientNetV2-S. EfficientNetV2-S is a convolutional neural network with eight primary backbone blocks [1]. The blocks consist of convolutional layers, squeeze and excitation blocks, and residual connections. We selected this model because it is specifically designed for image classification tasks, achieves strong classification performance, and trains efficiently. The authors report that EfficientNetV2-S achieved higher accuracy than ViT on the ImageNet ILSVRC2012 dataset while requiring significantly less training time.

The only architectural modification that we made to EfficientNetV2-S was changing the classification head. By default, the classification head of the model has 1,000 outputs, which we replaced with a fully connected layer with 100 outputs. We did not adjust or remove the dropout layer present in the model's default classification head. The final fine-tuning strategy involved unfreezing the entire model during training. This was made feasible in part due to the training efficiency of EfficientNetV2-S.

3.2 Training

Our training procedure uses cross-entropy loss with a label smoothing factor of 0.1. We also use the AdamW optimizer with a weight decay of $1e-4$. The classification head is trained with an initial learning rate of $1e-6$, while the model backbone is trained with an initial learning rate of $1e-7$. Both learning rates use a CosineAnnealingLR scheduler. We use a batch size of 16 and typically observe an accuracy plateau of approximately 80% after 100 epochs. After about 40 epochs, the model achieves over 70% accuracy, and further improvements require significantly more epochs.

Our model was trained on an NVIDIA GeForce RTX 3070 GPU. All training and testing runs were performed in a Docker container using Python 3.10, CUDA 12.6, PyTorch 2.12.0, Torchvision 0.27.0, and Torchaudio 2.11.0. Additional package version specifications can be found in the Dockerfile.

4. Experiments

To test multiple combinations of learning rates, models, unfreezing strategies, etc., we developed a testing framework using YAML configuration files. Our training script accepts a YAML configuration file which defines data, model, optimizer, and training options. This approach allows us to quickly iterate on different training methods and easily queue model training runs.

Our training process also systematically outputs run statistics into a structured output directory. Each run is assigned a timestamped UUID and its own output subdirectory. Runs are logged in a run manifest that tracks the run subdirectories and provides high-level run information. Each run subdirectory contains the YAML configuration used for training, a table of validation and train losses and accuracies across epochs, a summary of the best epoch, loss and accuracy curves, and a log file for the run. The manifest allows us to locate the corresponding run directory for training runs of interest. We were able to use the best-epoch

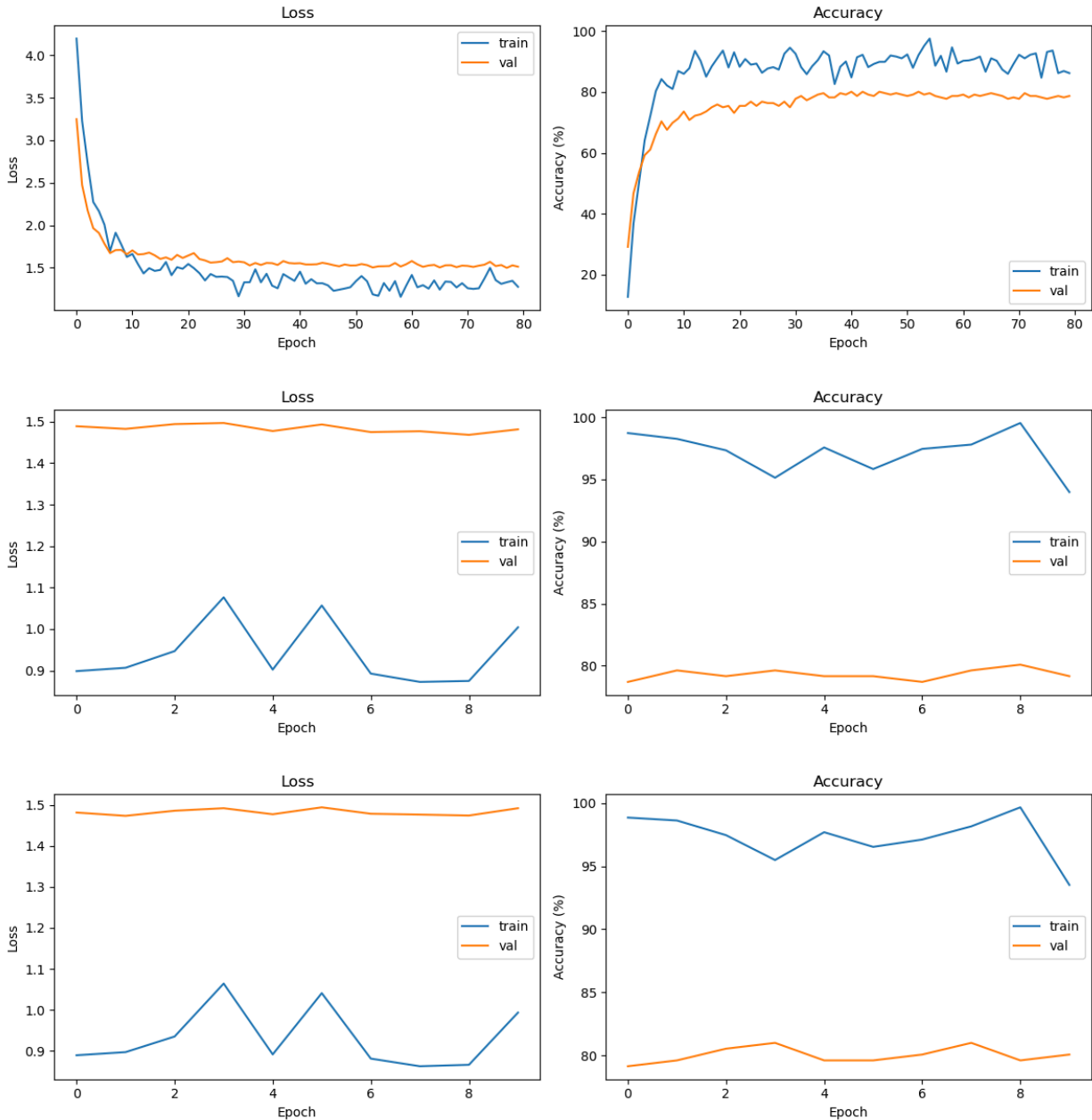
summary to quickly compare runs and determine if training was successfully improving accuracy.

Our experimentation began with small adjustments to the model and learning rates. We used low learning rates and unfroze a subset of the model layers. As we continued developing our training pipeline, we introduced simple data augmentation (flipping, rotation, cropping, color jitter, and greyscale conversion). To further improve accuracy, we added weight decay to the AdamW optimizer and introduced more advanced data augmentation techniques, including CutMix and MixUp.

5. Results

Across the training experiments conducted, the highest validation accuracy achieved was 81.02%, with the lowest validation loss of 1.49. We re-running the training, we obtained a validation accuracy of 78.70% and a validation loss of 1.56. Both results were achieved at approximately 100 epochs. However, after around 40 epochs the accuracy reaches approximately 70%, and subsequent epochs yield marginal improvements.

Our training was split into multiple runs. We did not train for more than 80 epochs at once in the interest of time and result monitoring. When additional epochs were needed, we loaded the best checkpoint from a previous run using our YAML configuration strategy to continue training. The following figures show our best training sequence: first 80 epochs of training, then selecting the best checkpoint and training for an additional 10 epochs, and finally selecting the best checkpoint from that run and training for another 10 epochs. The highest validation accuracy occurs in the third epoch of the final run corresponding to epoch 93 of total training. Our kaggle leaderboard score is 0.82727.



6. Discussion

The most impactful optimization in our training was data augmentation. The dataset used in this project is relatively small for machine learning applications, so it was important to generate synthetic variations of the data for model training. We began with basic augmentations, including flipping, rotation, cropping, color jitter, and greyscale conversion. Introducing more advanced augmentation techniques such as CutMix and MixUp further improved the mode's accuracy.

We also found success with unfreezing the entire model, but only for fine-tuning. We initialized low learning rates for both the classifier and the model backbone to ensure that training remained focused on fine-tuning. We initially trained with only the classifier unfrozen, which achieved approximately 50% accuracy. As we progressively unfroze layers of the backbone, accuracy increased to approximately 75%. When the entire model was unfrozen, we achieved our highest accuracy of approximately 80%.

We also attempted to initialize and train models other than EfficientNetV2-S. Most notably, we experimented with fully fine tuning ConvNeXtSmallTM [2]. This approach was not successful, as it resulted in lower accuracy compared to EfficientNetV2-S for similar training conditions.

An optimization we are interested in exploring is modifying the classification head for the model. In examining the dataset, it appears that there may be higher-level groupings of images, such as flowers, cars, airplanes, and desserts. We propose adding another classification head to EfficientNetV2-S to learn these broader categories. A second classifier that captures broader categories could help inform and improve the representations learned by the backbone for the primary 100-class classification task.

7. Reproducibility

7.1 Random Seed Configuration

The random seed for a training run can be set in the YAML configuration file under:

```
training:  
  seed:
```

For testing runs, the seed can be set using the `--seed` command-line argument. If not specified, the default seed value is 42.

7.2 Prerequisites

Docker must be installed on the machine.

Final project dataset must be placed in `./data`.

[Model weights used in final submission](#) must be placed in `./outputs`.

7.3 Environment Setup

First, clone the repository and navigate into it:

```
"git clone https://github.com/ellwi/cse144-final-project"  
"cd cse144-final-project"
```

Next, download the Kaggle dataset and place it in the root directory of the repository so that the structure matches:

```
cse144-final-project/  
  data/
```

train/
test/

Then build the Docker image:

```
"docker build -t cse144-final:latest ."
```

This step may take several minutes due to package installation. For exact package versions, refer to the Dockerfile.

7.4 Running Training

Run the following command to start training:

```
"docker run --rm --gpus all --shm-size=8g -v ./outputs:/app/outputs cse144-final:latest \
python train.py --config /app/configs/efficientnet/final_submission.yml"
```

After training completes, open:

```
./outputs/manifest.tsv
```

Find the run_id in the most recent training run (bottom row first column). If this is the first run on your machine, there will only be one entry.

7.5 Running Inference (Testing)

Run inference using the best checkpoint:

```
"docker run --rm --gpus all --shm-size=8g -v ./outputs:/app/outputs cse144-final:latest \
python inference.py --checkpoint /app/outputs/<run_id>/best_checkpoint.pth --outfile
/app/outputs/submission.csv"
```

Replace <run_id> with the run_id from manifest.tsv.

Output

The final submission file will be saved at:

```
./outputs/submission.csv
```

8. Team Contributions

Elina Wilson:

- Dataloaders and data augmentation
- EfficientNetV2-S and ConvNeXtSmallTM model implementations
- Training loop implementation
- Logging messages
- Presentation slides
- Presentation delivery

Piet Rottinghuis:

- Environment setup and reproducibility
- Interface for implementing transfer learning models
- Training, logging, output tracking
- Report writing
- Presentation delivery

References

- [1] M. Tan and Q. V. Le, "EfficientNetV2: Smaller Models and Faster Training," Jun. 23, 2021, *arXiv*: arXiv:2104.00298. doi: 10.48550/arXiv.2104.00298.
- [2] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, "A ConvNet for the 2020s," Mar. 02, 2022, *arXiv*: arXiv:2201.03545. doi: 10.48550/arXiv.2201.03545.